Color Threshold Adjustment and Color Block Calibration

By adjusting the HSV high and low thresholds, you can filter out interfering colors so that color blocks can be ideally recognized in complex environments. When using color-based features for the first time, it is recommended to calibrate. Each color-related AI feature in the 【scripts】 folder has its own 【HSV calibration HSV_calibration.ipynb】 file.

Here is the calibration file path for reference:

/dofbot_ws/src/dofbot_color_identify/scripts/HSV校准HSV_calibration.ipynb

HSV Introduction

HSV (Hue, Saturation, Value) is a color space created by A. R. Smith in 1978 based on the intuitive characteristics of colors, also known as the Hexcone Model. The parameters of this model are: Hue (H), Saturation (S), and Value (V).

H: 0 — 180

S: 0 — 255

V: 0 — 255

- HSV Parameter Table:

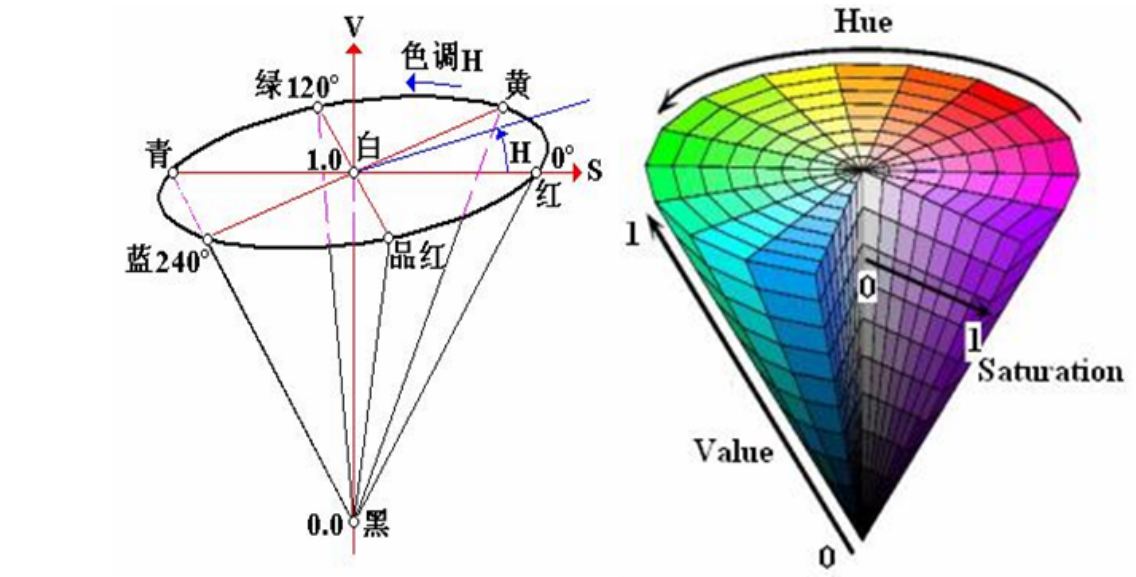
	Black	Gray	White	Red	Orange	Yellow	Green	Cyan	Blue	Purple	
H_min	0	0	0	0	156	11	26	35	78	100	125
H_max	180	180	180	10	180	25	34	77	99	124	155
S_min	0	0	0	43	43	43	43	43	43	43	
S_max	255	43	30	255	255	255	255	255	255	255	
V_min	0	0	0	46	46	46	46	46	46	46	
V_max	46	220	255	255	255	255	255	255	255	255	

- HSV Hexagonal Pyramid

(1) Hue (H). Represents color information, i.e., the position of the spectral color. This parameter is represented by an angular measure with a range of 0° to 360°, calculated counterclockwise starting from red at 0°, green at 120°, and blue at 240°. Their complementary colors are: yellow at 60°, cyan at 180°, and purple at 300°.

(2) Saturation (S). Saturation represents the ratio between the purity of the selected color and the maximum purity of that color. When S=0, there is only grayscale. Every 120 degrees apart, complementary colors differ by 180 degrees. A color can be considered as a mix of a spectral color and white. The greater the proportion of the spectral color, the closer the color is to the spectral color, and the higher the saturation. High saturation means deeper and more vivid colors. The spectral color's white component is 0, reaching maximum saturation. Usually the range is 0% to 100%, with higher values meaning more saturated colors.

(3) Value (V). Value represents the brightness of the color. For light source colors, the value is related to the luminance of the light source. For object colors, this value is related to the transmittance or reflectance of the object. Usually the range is 0% (black) to 100% (white). Note: There is no direct connection between this and light intensity. The 3D representation of the HSV model evolves from the RGB cube. Imagine observing from the white vertex along the diagonal of the RGB cube to the black vertex, and you can see the hexagonal shape of the cube. The hexagon boundary represents colors, the horizontal axis represents purity, and value is measured along the vertical axis.



Code Design

- Import header files

```
import threading
import cv2 as cv
from time import sleep
from dofbot_config import *
import ipywidgets as widgets
from IPython.display import display
```

- Create instance, initialize parameters

```
# Create update HSV instance
update_hsv = update_hsv()
# Initialize num parameter
num=0
# Initialize mode
model = "General"
# Initialize HSV name
HSV_name=None
# Initialize HSV values
color_hsv = {"red" : ((0, 143, 163), (11, 255, 255)),

        "green" : ((55, 80, 66), (78, 255, 255)),

        "blue" : ((110, 100, 121), (117, 255, 255)),

        "yellow": ((26, 100, 91), (32, 255, 255))}
```

```
# HSV parameter path (jupyter lab and ros package need absolute path)
HSV_path="/home/jetson/dofbot_ws/src/dofbot_color_identify/scripts/HSV_config.tx
t"
# Read HSV config file, update HSV values
try: read_HSV(HSV_path,color_hsv)
except Exception: print("No HSV_config file!!!")
```

- Create controls

```
button_layout = widgets.Layout(width='320px', height='55px',
align_self='center')
output = widgets.Output()
# Enter color update mode
HSV_update_red= widgets.Button(description='HSV_update_red',
button_style='success',layout=button_layout)
...
# Adjustment sliders

H_min_slider=widgets.IntSlider(description='H_min
:',value=0,min=0,max=255,step=1, orientation='horizontal')
...
# Exit
exit_button=widgets.Button(description='Exit',button_style='danger',layout=butto
n_layout)
```

- Color update callbacks

```
def   update_red_Callback(value):
    pass

def   update_green_Callback(value):
    pass

def   update_blue_Callback(value):
    pass

def   update_yellow_Callback(value):
    pass

HSV_update_red.on_click(update_red_Callback)
HSV_update_green.on_click(update_green_Callback)
HSV_update_blue.on_click(update_blue_Callback)
HSV_update_yellow.on_click(update_yellow_Callback)
```

- Mode switching controls

```
def   write_file_Callback(value):
    pass

def   Color_Binary_Callback(value):
    pass

def   exit_button_Callback(value):
    pass

HSV_write_file.on_click(write_file_Callback)
Color_Binary.on_click(Color_Binary_Callback)
exit_button.on_click(exit_button_Callback)
```

```
For detailed code, see dofbot_ws/src/dofbot_color_identify/scripts/HSV校准.ipynb
```

- Interface example

The upper center of the screen displays which color to select. The six sliders on the upper right correspond to the six HSV values. Slide the sliders to adjust each color's HSV threshold in real-time.

Operation Process

(1). After starting all code blocks, the interface shown at the bottom of the code will be displayed. By default, no color is selected, so no colors are recognized.

(2). Click the 【HSV_update_green】 button to start recognizing green objects (Note: This color recognition detects contours, so the object must be completely within the camera range for normal recognition). Slide the sliders on the upper right to adjust the green HSV threshold. Pay attention to adjusting from multiple angles and environments until the object can be clearly recognized in complex environments without being disturbed by other objects 【Other colors are similar】.

(3). The 【Color/Binary】 button switches between color image and binary image. It only works after selecting a color. After switching, only the selected color's binary image is displayed, making it easier for us to debug.

(4). After debugging all colors' HSV values, click the 【HSV_write_file】 button. All debugged parameters will be saved in 【.txt】 format in the same path as the code. The next startup will automatically read the parameters from the file.

(5). The 【Exit】 button closes the camera and exits the program.